

IntegriX

AI 기반 텍스트.이미지 분석 및 피싱 위험 URL 탐지 플랫폼

개인 프로젝트 포트폴리오

이름: 홍길동

GitHub: <https://github.com/본인아이디>

Notion: <https://notion.so/본인노션주소>

Email: 본인이메일@example.com

자기소개

서비스 전체 흐름을 이해하는 풀스택 개발자를 목표로 합니다.

관심 분야

- Spring Boot 기반 백엔드 API 개발
- 보안 및 인증 처리
- 외부 API 연동
- AI API를 활용한 분석 서비스
- 클라우드 배포 및 운영

보유 기술

- Backend: Java 17, Spring Boot, Spring Security, JWT
- Database: MariaDB, MongoDB Atlas, Redis
- Frontend: React, TypeScript, Vite, Axios
- Infra: AWS EC2, AWS RDS, Nginx
- API: Google Safe Browsing, RDAP, OpenAI API

프로젝트명 및 소개

IntegriX는 온라인 위험 요소를 하나의 서비스에서 분석합니다.

- URL, 텍스트, 이미지를 분석해 피싱 URL, 가짜뉴스 가능성, 이미지 조작 가능성을 탐지하는 웹 서비스입니다.
- 사용자는 의심스러운 URL, 문장, 이미지를 입력하고 위험도와 분석 근거를 확인할 수 있습니다.
- 분석 결과는 MongoDB에 저장되며, 대시보드와 분석 이력 화면에서 다시 확인할 수 있습니다.
- 신고, 관리자 관리, 공지사항 기능을 함께 구현해 서비스 운영 흐름까지 구성했습니다.

문제점 및 제안 이유

사용자가 온라인 콘텐츠의 위험성을 직접 판단하기 어렵다는 문제에서 출발했습니다.

- 피싱 URL, 허위 정보, 조작 이미지는 겉보기에는 정상 콘텐츠처럼 보일 수 있습니다.
- URL, 텍스트, 이미지 분석 도구가 각각 분리되어 있어 사용자가 여러 서비스를 따로 이용해야 합니다.
- 단순히 안전 또는 위험만 보여주면 사용자가 결과를 신뢰하기 어렵습니다.
- 분석 결과와 판단 근거를 함께 제공하고, 분석 기록을 한곳에서 관리할 수 있는 서비스가 필요하다고 판단했습니다.

해결방안

자체 규칙, 외부 보안 API, AI 분석을 결합했습니다.

URL 분석

- URL 정규화 및 형식 검증
- DNS, HTTPS, SSL 상태 검사
- Google Safe Browsing API 연동
- RDAP 기반 도메인 등록 정보 분석

AI 분석

- OpenAI API 기반 텍스트 분석
- 가짜뉴스 가능성 판단
- 이미지 조작 가능성 분석
- 분석 결과 및 상세 근거 저장

운영 기능

- 분석 이력 및 대시보드
- 신고 및 관리자 처리
- 공지사항 관리
- JWT 인증 및 배포 환경 구성

본인이 구현한 기능

개인 프로젝트로 기획부터 배포까지 전체 기능을 직접 구현했습니다.

- Spring Boot 기반 URL, 텍스트, 이미지 분석 REST API 구현
- URL 위험도 점수 계산 로직 및 scoreFactors 기반 판단 근거 제공
- Google Safe Browsing API, RDAP, OpenAI API 연동
- MariaDB 기반 회원, 신고, 공지사항 데이터 관리
- MongoDB 기반 분석 결과 및 상세 이력 저장
- React + TypeScript 기반 분석 화면, 대시보드, 마이페이지, 관리자 페이지 구현
- AWS EC2, RDS, MongoDB Atlas, Redis, Nginx 기반 배포 환경 구성

핵심 구현 사례: URL 위험도 분석

여러 위험 신호를 점수화하고, 결과의 근거를 함께 제공했습니다.

- DNS 조회 실패, SSL 인증서 오류, HTTPS 연결 실패 여부를 검사했습니다.
- 긴 URL, @ 문자 포함, 하이픈 과다 사용, IP 직접 사용, 단축 URL 여부를 점수화했습니다.
- Google Safe Browsing에서 위험 URL로 확인되면 위험 점수를 크게 반영했습니다.
- RDAP 조회로 도메인 생성일을 확인하고 최근 등록 도메인일 경우 위험 요소로 반영했습니다.
- 최종 점수뿐 아니라 어떤 요소가 점수에 영향을 주었는지 scoreFactors로 제공했습니다.

시스템 구성

프론트엔드, 백엔드, 데이터 저장소, 외부 API를 분리해 구성했습니다.

Frontend

- React
- TypeScript
- Vite
- Axios

Backend

- Spring Boot
- Spring Security
- JWT
- REST API

Data

- MariaDB
- MongoDB Atlas
- Redis

External / Infra

- Google Safe Browsing
- RDAP
- OpenAI API
- AWS EC2, RDS, Nginx

GitHub 및 화면 캡처

저장소 링크와 주요 화면 이미지를 추가하는 페이지입니다.

- Backend GitHub: <https://github.com/본인아이디/IntegriX>
- Frontend GitHub: <https://github.com/본인아이디/IntegriX-Frontend>
- 추가 권장 화면: 대시보드, URL 분석 결과, 텍스트 분석 결과, 이미지 분석 결과, 관리자 페이지, 공지사항 화면
- 실제 제출 전 본인 GitHub 링크와 화면 캡처 이미지를 교체하면 됩니다.

배운 점

기능 구현을 넘어 분석 근거와 운영 흐름까지 고려했습니다.

- 외부 API, 인증, 데이터 저장소, 배포 환경을 함께 구성하며 서비스 전체 흐름을 경험했습니다.
- URL 위험도 분석에서 여러 신호를 조합해 점수를 계산하고, 사용자가 이해할 수 있는 근거를 제공하는 방법을 배웠습니다.
- 데이터 성격에 따라 MariaDB와 MongoDB를 나누어 사용하는 경험을 했습니다.
- 앞으로도 사용자가 신뢰할 수 있는 결과를 제공하는 서비스를 개발하고 싶습니다.